CSED233 · 데이터구조 · 포스텍

시험 전에 볼 것

핵심 요약 치트시트 · 연산 빠른 복습 · Pseudocode(의사코드) 모음. 자세한 설명은 각 정리본으로.

1. 개념 치트

– Circular Queue(원형 큐) — modulo와 Full/Empty 구분

일반 배열 큐는 Dequeue/Enqueue를 반복하면 Front/Rear가 오른쪽으로만 밀려, 앞이 비어도 Rear가 끝에 닿으면 삽입 불가. 배열의 끝과 처음을 논리적으로 연결해 해결한다.

- **인덱스 갱신**: `Rear = (Rear + 1) % size` , `Front = (Front + 1) % size`

Empty와 Full이 둘 다 Front == Rear

해결책 둘 중 하나를 쓴다:

- **1칸 비워두기**(가장 흔함): `(Rear + 1) % size == Front` 이면 Full로 간주. 여기서 front는 첫 원소 자체가 아니라 첫 원소의 한 칸 앞을 가리키는 규약을 쓴다. 그래서 비어 있을 때도 front == rear이고, 원소가 하나 들어간 뒤에도 front는 그 원소보다 한 칸 앞에 머문다.
- **count 변수**로 데이터 개수를 별도 추적.

– Tree 재구성(Reconstruction) — 어떤 순회 조합이 가능한가

단일 순회 하나만으로는 원본 트리를 재구성할 수 없다. Inorder를 포함한 조합이면 가능 — Inorder가 왼쪽/오른쪽 구간의 경계를 알려주기 때문.

단, 아래의 '가능'은 키가 서로 다르다는 전제에서의 재구성을 뜻한다. 중복 키가 있으면 Inorder에서 같은 키가 어느 쪽 구간에 속하는지가 모호해질 수 있다.

- ✓ Preorder + Inorder · ✓ Postorder + Inorder · ✓ Level-order + Inorder
- ✗ Preorder + Postorder
- Preorder + Postorder만으로는 안 되는 이유는, 두 순회 모두 루트는 알 수 있지만 Inorder처럼 왼쪽 서브트리와 오른쪽 서브트리의 경계를 나눠 주는 정보가 없어서, 같은 두 순회를 만족하는 서로 다른 트리가 존재할 수 있기 때문이다.

예제 · Pre: A B D E C, In: D B E A C

① Preorder의 첫 원소 **A**가 루트.

② Inorder에서 A의 위치로 분할: 왼쪽 `[D B E]`, 오른쪽 `[C]`.

③ 각 구간을 재귀 처리 → 최종 트리:

```
  A
 /\
B  C
/\
D  E
```

– General Tree(일반 트리) 구현 3가지 + 이진 트리 변환

방식	구조	장점	단점
Simple(부모 배열)	노드마다 부모 인덱스 저장	Parent(k) $O(1)$	자식 찾기 $O(n)$ (전체 스캔), 자식 순서 정보 없음

List-of-Children	노드마다 자식 연결 리스트	두 트리 결합이 $O(1)$(루트의 부모만 갱신)	오른쪽 형제 접근 어려움, 데이터 중복 저장
Left-Child / Right-Sibling(LCRS)	포인터 2개: 첫째 자식 + 다음 형제	자식 수와 무관하게 고정 포인터 2개 → 어떤 다항 트리든 이진 트리로 표현	—

일반 트리 → 이진 트리 변환

① 수직 링크 = Left child, 수평 링크 = Right sibling으로 다시 연결.

② 시계 방향 45° 회전하면 이진 트리 완성.

```
원본:     a        변환 후:   a
        / | \                /
       b  c  d              b
      / \                  / \
     e   f                e   c
                         \   \
                          f   d
```

순회 대응 (시험 단골): 일반 트리의 **Preorder** = 변환 이진 트리의 **Preorder**, 일반 트리의 **Postorder** = 변환 이진 트리의 **Inorder**. (위 예: Preorder a b e f c d / Postorder e f b c d a = 변환 트리 Inorder)

– Pointer(포인터) 기본기

```
 int x = 10;
int* p = &x;   // pointer 선언 + 주소 저장 (int* p와 int *p는 동일)

*p            // p가 가리키는 메모리의 값 (= 10) — 역참조
*p = 20       // x의 값을 20으로 변경

struct node { int value; node* next; };
node* q = new node;   // 노드 생성
q->value = 5;         // 멤버 접근은 ->
q->next = nullptr;
```

– Big-O · Big-Omega · Big-Theta

Big-O(상한): 어떤 양의 상수 c>0와 기준점 n_0가 존재해서, $n \geq n_0$인 모든 n에 대해 $T(n) \leq cf(n)$를 만족하면 T(n) = O(f(n))이다. 즉 충분히 큰 입력에서는 실행 시간이 f(n)에 상수배를 곱한 값을 넘지 않는다는 상한을 뜻한다.

$$T(n) \leq c\, f(n) \quad (n \geq n_0)$$

Big-Omega(하한): 어떤 양의 상수 c>0와 기준점 n_0가 존재해서, $n \geq n_0$인 모든 n에 대해 $T(n) \geq cf(n)$를 만족하면 T(n) = Ω(f(n))이다. 즉 충분히 큰 입력에서는 실행 시간이 f(n)에 상수배를 곱한 값보다 작아지지 않는다는 하한을 뜻한다.

$$T(n) \geq c\, f(n) \quad (n \geq n_0)$$

Big-Theta(동등): 어떤 양의 상수 c_1, c_2와 기준점 n_0가 존재해서, $n \geq n_0$인 모든 n에 대해 $c_1 f(n) \leq T(n) \leq c_2 f(n)$를 만족하면 T(n) = Θ(f(n))이다. 즉 상한과 하한이 같은 f(n)으로 동시에 묶여, 증가 정도가 f(n)과 동등하다는 뜻이다.

$$c_1 f(n) \leq T(n) \leq c_2 f(n) \quad (n \geq n_0)$$

2. 연산 빠른 복습

– Heap — 두 성질 · Insert/DeleteMin · Heapify O(n)

- ① Structural: Complete Binary Tree · ② Heap Order: Min-Heap 부모 ≤ 자식 / Max-Heap 부모 ≥ 자식.
- **Insert**: 마지막에 넣고 부모와 비교하며 **위로** swap — $O(\log n)$.
- **DeleteMin**: 마지막 노드를 루트에 놓고 **더 작은 자식**과 비교하며 아래로 swap — $O(\log n)$.
- **Heapify**: 마지막 비단말 노드($\lfloor n/2 \rfloor$)부터 루트까지 역순 Down-heap — $O(n)$.
- **Heapify가 O(n)인 이유**: Down-heap 비용이 노드 높이에 비례하는데 높이가 큰 노드는 적고 낮은 노드는 많기 때문이다. 높이가 h인 노드는 최대 $\lceil n/2^{h+1} \rceil$개이고 각 노드의 Down-heap 비용은 O(h)이므로, 전체 작업량은 $\Sigma\, (n/2^{h+1}) \cdot h$ (h에 대한 합) 형태로 묶여 O(n)이 된다.
- 참고로 §3 의사코드의 Heap push/pop은 Max-Heap 기준이며, 여기서 pop이 최댓값을 제거하는 연산으로 위 요약의 DeleteMin과 대응한다(Min-Heap에서는 같은 구조로 DeleteMin이 된다).

$$\sum_{k=0}^{h} 2^k (h-k) \;\le\; n \sum_{j=0}^{h} \frac{j}{2^j} \;\le\; 2n \;=\; O(n)$$

자세한 그림과 증명 → 정리본 7. Heaps

– Sorting — 복잡도 표와 Stability

- 단순 정렬(Bubble/Selection/Insertion): $O(n^2)$ · 고급 정렬(Merge/Quick/Heap): $O(n \log n)$
- **안정 정렬**(같은 값의 순서 유지): Merge, Insertion, Bubble — 인접 비교/병합 방식.
- **불안정 정렬**: Quick, Heap, Selection — pivot/힙 기반으로 멀리 떨어진 원소를 교환.

알고리즘	최선	평균	최악	공간	안정
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓

Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ · 최악 O(n)	✗
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗

Quick Sort의 최악 O(n^2)은 pivot이 매 단계 가장 작거나 가장 큰 원소처럼 치우쳐서, 분할이 한쪽에 원소 1개(또는 0개)와 나머지 전체로 계속 불균형하게 나뉠 때 발생한다. 이때 재귀 깊이가 n까지 깊어지고 전체 비교 횟수가 1+2+⋯+n 수준으로 누적된다.

분할·병합·파티셔닝 예제 → 정리본 8. Sorting

– BST — 성질 · Delete 3 cases

- 왼쪽 서브트리 < 노드 < 오른쪽 서브트리 · **Inorder = 오름차순**.
- Search/Insert/Delete 모두 O(높이): 균형 $O(\log n)$, 편향 $O(n)$.
- **Delete**: ① leaf → 제거 · ② 자식 1개 → 자식으로 대체 · ③ 자식 2개 → **Successor(오른쪽 서브트리의 최소원소)**로 대체 후 successor 삭제. Successor는 오른쪽 서브트리에서 가장 작은 키라 왼쪽 자식이 없으므로, 자식이 최대 1개(오른쪽 자식만)뿐이라 ①/②로 환원된다.

예제와 그림 → 정리본 9. Binary Search Tree

– AVL Tree — bf · 회전 4유형 · Insert vs Delete

- AVL = BST + Height-Balanced: 모든 노드 $-1 \leq bf(x) \leq 1$. bf(x) = h(왼쪽) − h(오른쪽).
- **LL** → 우회전 · **RR** → 좌회전 · **LR** → 자식 좌회전 + 부모 우회전 · **RL** → 자식 우회전 + 부모 좌회전.
- 회전 후에도 BST 성질(inorder 순서) 유지.
- **Insert는 최대 1회 회전, Delete는 연쇄 회전 가능**(회전으로 높이가 줄면 위로 전파).

회전 메커니즘과 연쇄 불균형 예제 → 정리본 10. AVL Tree

3. Pseudocode 모음

– Array List — insert / erase

```
METHOD insert(index, element) {
    IF index < 0 OR index > size THEN THROW OutOfBoundsException
    IF size == capacity THEN resize(capacity * 2)

    // 데이터를 한 칸씩 뒤로 밀기 (Right Shift)
    FOR i FROM size DOWNTO index + 1 DO
        data[i] ← data[i - 1]
    data[index] ← element
    size ← size + 1
}

METHOD erase(index) {
    IF index < 0 OR index >= size THEN THROW OutOfBoundsException
    removedValue ← data[index]

    // 데이터를 한 칸씩 앞으로 당기기 (Left Shift)
    FOR i FROM index TO size - 2 DO
        data[i] ← data[i + 1]
    size ← size - 1
    RETURN removedValue
}
```

– Linked List (dummy head) — insert / removeAt

왜 dummy?

맨 앞 삽입·삭제도 예외 없이 같은 코드로 처리하려고, 데이터를 담지 않는 빈 head 노드를 둔다.
`prev` 는 항상 존재한다.

```
CONSTRUCTOR() {
    head ← NEW Node()      // 더미 노드
    head->next ← NULL
    size ← 0
}

METHOD insert(index, value) {
    IF index < 0 OR index > size THEN RETURN ERROR
    prev ← head
    FOR i FROM 0 TO index - 1 DO prev ← prev->next   // 삽입 위치 바로 앞

    newNode ← NEW Node()
```

```
    newNode->data ← value
    newNode->next ← prev->next
    prev->next ← newNode
    size ← size + 1
}

METHOD removeAt(index) {
    IF index < 0 OR index >= size THEN RETURN ERROR
    prev ← head
    FOR i FROM 0 TO index - 1 DO prev ← prev->next

    target ← prev->next
    prev->next ← target->next
    DELETE target
    size ← size - 1
}
```

– Stack — Array / Linked

```
// Array Stack: top = 가장 위 원소의 인덱스 (비면 -1), MAX_SIZE = 배열 용량(최대 원소 개수)
METHOD push(stack, value) {
    IF stack.top == MAX_SIZE - 1 THEN SIGNAL "Stack Overflow"
    stack.top ← stack.top + 1
    stack.data[stack.top] ← value
}
METHOD pop(stack) {
    IF stack.top == -1 THEN SIGNAL "Stack Underflow"
    value ← stack.data[stack.top]
    stack.top ← stack.top - 1
    RETURN value
}
```

```
// Linked Stack: top이 첫 노드를 가리킴
METHOD push(value) {
    newNode ← NEW Node()
    newNode->data ← value
    newNode->next ← top     // 기존 top을 아래로
    top ← newNode
    size ← size + 1
}
METHOD pop() {
    IF top == NULL THEN SIGNAL "Stack Underflow"
```

```
    temp ← top
    value ← temp->data
    top ← top->next
    DELETE temp            // C++에서 메모리 해제 필수
    size ← size - 1
    RETURN value
}
```

– Queue — Circular(Array) / Linked

```
 // Circular Queue: front는 첫 요소의 "한 칸 앞", 1칸 비워두기 방식
METHOD enqueue(queue, value) {
    IF (queue.rear + 1) % queue.capacity == queue.front THEN
        SIGNAL "Queue Overflow"
    queue.rear ← (queue.rear + 1) % queue.capacity
    queue.data[queue.rear] ← value
}
METHOD dequeue(queue) {
    IF queue.front == queue.rear THEN SIGNAL "Queue Underflow"
    queue.front ← (queue.front + 1) % queue.capacity
    RETURN queue.data[queue.front]
}
```

```
 // Linked Queue: front(나가는 쪽), rear(들어오는 쪽)
METHOD enqueue(value) {
    newNode ← NEW Node()
    newNode->data ← value
    newNode->next ← NULL
    IF rear == NULL THEN front ← newNode; rear ← newNode   // 빈 큐
    ELSE rear->next ← newNode; rear ← newNode
    size ← size + 1
}
METHOD dequeue() {
    IF front == NULL THEN SIGNAL "Queue Underflow"
    temp ← front
    value ← temp->data
    front ← front->next
    IF front == NULL THEN rear ← NULL    // 비게 된 경우 rear도 정리
    DELETE temp
    size ← size - 1
```

```
    RETURN value
}
```

Linked Queue의 dequeue에서 마지막 노드를 빼면 **rear도 NULL로** 갱신해야 한다. 안 하면 rear가 해제된 노드를 가리킨다.

– Tree — Array 구현 / Linked + 순회 / DFS(스택) / BFS(큐)

배열 인덱스 관계 (1-indexed)

왼쪽 자식 `2i` · 오른쪽 자식 `2i+1` · 부모 `⌊i/2⌋`

```
METHOD insert(value) {              // 완전 이진 트리 순서로 삽입
    IF size + 1 >= capacity THEN SIGNAL "Tree Full"
    size ← size + 1
    data[size] ← value
}
METHOD getLeftChild(index) {
    left ← index * 2
    IF left > size THEN RETURN -1     // 자식 없음
    RETURN data[left]
}
METHOD getParent(index) {
    IF index <= 1 THEN RETURN -1      // 루트는 부모 없음
    RETURN data[index / 2]
}
```

```
// Linked tree 순회 (재귀)
METHOD preOrder(node) {
    IF node == NULL THEN RETURN
    PRINT node->data                // 1. 현재
    preOrder(node->left)            // 2. 왼쪽
    preOrder(node->right)           // 3. 오른쪽
}
METHOD inOrder(node) {
    IF node == NULL THEN RETURN
    inOrder(node->left)
    PRINT node->data
    inOrder(node->right)
}
```

```
// DFS — 재귀 없이 스택으로
METHOD DFS_Stack(startNode) {
    IF startNode == NULL THEN RETURN
    Stack s
    s.push(startNode)
    WHILE s is not empty DO
        curr ← s.pop()
        PRINT curr->data
        // 오른쪽을 먼저 넣어야 왼쪽이 먼저 나온다 (LIFO)
        IF curr->right != NULL THEN s.push(curr->right)
        IF curr->left  != NULL THEN s.push(curr->left)
    END WHILE
}

// BFS — 큐
METHOD BFS(root) {
    IF root == NULL THEN RETURN
    Queue q
    q.enqueue(root)
    WHILE q is not empty DO
        curr ← q.dequeue()
        PRINT curr->data
        IF curr->left  != NULL THEN q.enqueue(curr->left)
        IF curr->right != NULL THEN q.enqueue(curr->right)
    END WHILE
}
```

DFS = **Stack**(또는 재귀 호출 스택), BFS = **Queue**. preOrder/inOrder가 곧 DFS의 한 형태다.

- Heap — push / pop (array, Max-Heap)

```
METHOD push(heap, value) {
    heap.size ← heap.size + 1
    curr ← heap.size
    heap.data[curr] ← value
    // 부모보다 크면 위로 교체 (Up-heap)
    WHILE curr > 1 AND heap.data[curr/2] < heap.data[curr] DO
        swap(heap.data[curr/2], heap.data[curr])
        curr ← curr / 2
    END WHILE
}
```

```
METHOD pop(heap) {
  IF heap.size == 0 THEN RETURN ERROR
  maxValue ← heap.data[1]
  heap.data[1] ← heap.data[heap.size]   // 마지막을 루트로
  heap.size ← heap.size - 1

  curr ← 1
  WHILE (curr * 2) <= heap.size DO
    child ← curr * 2
    // 더 큰 자식 선택
    IF child + 1 <= heap.size AND heap.data[child] < heap.data[child+1] THEN
      child ← child + 1
    END IF
    IF heap.data[curr] >= heap.data[child] THEN BREAK
    swap(heap.data[curr], heap.data[child])
    curr ← child
  END WHILE
  RETURN maxValue
}
```

- Sort — Merge / Quick / Quick(in-place)

```
Merge-sort(L, n):
  if n == 1: return L                  // 기저 조건
  L1 = L의 앞 절반, L2 = L의 뒤 절반        // 분할
  return merge(Merge-sort(L1, length(L1)), Merge-sort(L2, length(L2)))  // n이 홀수면 두 부분 길이가 다름
```

```
Quicksort(A):
  if length(A) <= 1 then return A
  pivot 선택 후 A에서 제거
  for each x in A:
    if x < pivot then append x to L
    else append x to R
  return concatenate(Quicksort(L), pivot, Quicksort(R))
```

```
Quicksort(A, low, high):
  if low < high:
    p = Partition(A, low, high)
    Quicksort(A, low, p - 1)      // pivot은 확정 — 범위에서 제외
    Quicksort(A, p + 1, high)
```

```
Partition(A, low, high):
  pivot = A[low]
  leftwall = low                  // pivot 미만 구역의 끝
  for i = low + 1 to high:
    if A[i] < pivot:
      leftwall = leftwall + 1    // 구역 확장 뒤 교환
      swap(A[i], A[leftwall])
  swap(A[low], A[leftwall])       // pivot을 경계로
  return leftwall
```

– BST — search / insert / delete (CLRS 스타일)

```
search(x, k):
  if x == NIL or k == x.key: return x
  if k < x.key: return search(x.left, k)
  else:       return search(x.right, k)
```

```
insert(T, z):
  y = NIL; x = T.root
  while x != NIL:             // 삽입 위치의 부모 y 찾기
    y = x
    if z.key < x.key: x = x.left
    else:           x = x.right
  z.parent = y
  if y == NIL:       T.root = z    // 빈 트리
  elif z.key < y.key: y.left = z
  else:              y.right = z
```

```
delete(T, z):
  if z.left == NIL:                // 자식 0개 또는 오른쪽만
    transplant(T, z, z.right)
  else if z.right == NIL:          // 왼쪽만
    transplant(T, z, z.left)
  else:                            // 자식 둘
    y = minimum(z.right)           // Successor
    if y.parent != z:
      transplant(T, y, y.right)
      y.right = z.right
      y.right.parent = y
    transplant(T, z, y)
```

```
        y.left = z.left
        y.left.parent = y
```

`transplant(T, u, v)` 는 u의 자리를 서브트리 v로 통째로 교체하는 도우미. Degree-2 삭제에서 successor y가 z의 직계 자식이 아니면, y를 먼저 빼내고(①/② 환원) z 자리에 심는 두 단계로 나뉜다.

– General Tree → LCRS 변환 코드

```
STRUCT BinaryNode {
    INT data;
    BinaryNode* left;   // 원래의 첫째 자식
    BinaryNode* right;  // 원래의 다음 형제
}

METHOD convertToBinary(gNode) {
    IF gNode == NULL THEN RETURN NULL
    bNode ← NEW BinaryNode(gNode->data)

    IF gNode.children.isNotEmpty() THEN
        bNode->left ← convertToBinary(gNode.children[0])   // 첫째 자식

    // 나머지 자식들을 형제 체인으로 연결
    curr ← bNode->left
    FOR i FROM 1 TO gNode.children.size() - 1 DO
        curr->right ← convertToBinary(gNode.children[i])
        curr ← curr->right
    RETURN bNode
}
```